Name : K. Archana course : Data Structures

Reg :- 192524209 code : CSA0310

CO2 AT 1

1) Identify if the given expression are balanced or not

(i) (K+8) - {6+e}+77) /100

Expression

[q* (K+8) - {6+e}+77) /100

symbol	stack operation
[	push [
(	push (
)	pop (✓
{	push {
}	pop { ✓
End of Expression	[remains in stack ✗

→ Result Not Balance

Reason: The opening square bracket [does not have a matching close bracket]

(ii) Expression :

(((a*b)+[c/2]-d)

symbol	stack operation
(	push
(	push
(	push
)	pop √
[	push
]	pop √
)	pop √
End of expression	Two (remain in stack

Result: Not Balance

Reason: Two opening parenthese (are not closed

Algorithm to check Balanced Parentheses:

→ If the opening symbol ((, {, [ ) is found, push it onto the stack.

→ If a closing symbol ( ), },]) is found.

→ If the stack is empty →, Not Balance.

→ otherwise pop the top element.

Time complexity : O(n)

Space complexity : O(n)

2) Stack compression (value & frequency)

Problem:-

A stack contains repeated elements.

Example:

Original stack (Top → Bottom):

5
5
5
3
2
2
1

Instead of storing duplicates separately each value with its frequency.

Compressed stack.

Value	Frequency
5	3
3	2
2	3
1	1

Modified Stack ADT:

Node
value
frequency

Push (x)

→ If stack empty → Insert(x,1)

→ top equals x → Increase

Pop ()

frequency > 1 → Decrease

else → Remove node

Peek ()

Returns the value at the top of the stack

Display()

Print each elements alongs its frequency.

Example :-

5 x 3

3 x 2

2 x 3

1 x 1

original stack :-

Top :-

5

5

5

3

3

2

2

1

After compression :-

(5, 3)

(3, 2)

(2, 3)

(1, 1)

Modified stack ADT Diagram

Top

value = 5

value = 3

↓

value = 3

count = 2

↓

value = 2

count = 3

↓

value = 1

count = 1

↓

NULL

Trade-of between Time and space

Parameter	Normal stack	compressed stack
Memory usage	High	Low
push	O(1)	O(1)
Pop	O(1)	O(1)
Peek	O(1)	O(1)
Extra peek	No	-frequency
Best used	Distinct	Repeated

Analysis :

Space efficiency; Memory is saved because repeated elements are stored only once with a frequency count.

Time efficiency : Push , POP and Peek remain O(1)

Trade - off : A small amount of extra memory is required for storing the frequency count but significant memory is saved when many duplicate elements are present.

conclusion :-

stack compression using (value, frequency) reduces memory usage while preserving O(1) stack operation.

→ This provides a better space - efficient implementation with minimal additional overhead.